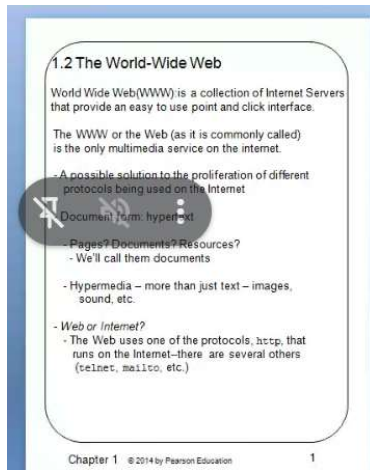


Web Programming-UNIT-I

16 July 2026 18:32



Screen clipping taken: 16

What is HTML?

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

A Simple HTML Document

Example

```
<!DOCTYPE html>

<html>

<head>

<title>Page Title</title>

</head>

<body>

<h1>My First Heading</h1>

<p>My first paragraph.</p>

</body>

</html>
```

Example Explained

- The `<!DOCTYPE html>` declaration defines that this document is an HTML5 document

- The `<html>` element is the root element of an HTML page
- The `<head>` element contains meta information about the HTML page
- The `<title>` element specifies a title for the HTML page (which is shown in the browser's title bar or in the page's tab)
- The `<body>` element defines the document's body, and is a container for all the visible contents, such as headings, paragraphs, images, hyperlinks, tables, lists, etc.
- The `<h1>` element defines a large heading
- The `<p>` element defines a paragraph

Note: Some HTML elements have no content (like the `<br>` element). These elements are called empty elements. Empty elements do not have an end tag!

What is an HTML Element?

An HTML element is defined by a start tag, some content, and an end tag:

`<tagname>` Content goes here... `</tagname>`

The HTML **element** is everything from the start tag to the end tag:

`<h1>`My First Heading`</h1>`

`<p>`My first paragraph.`</p>`

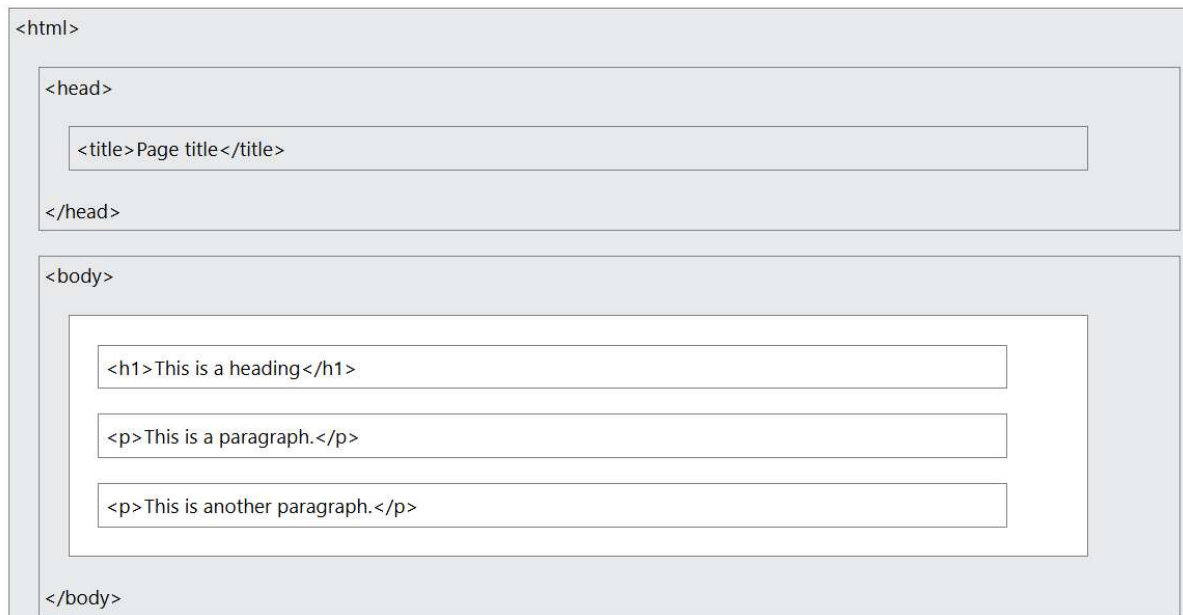
Start tag	Element content	End tag
<code><h1></code>	My First Heading	<code></h1></code>
<code><p></code>	My first paragraph.	<code></p></code>
<code>
</code>	<i>none</i>	<i>none</i>

Web Browsers

The purpose of a web browser (Chrome, Edge, Firefox, Safari) is to read HTML documents and display them correctly. A browser does not display the HTML tags, but uses them to determine how to display the document:

HTML Page Structure

Below is a visualization of an HTML page structure:



Screen clipping taken: 29-07-2026 16:00

Note: The content inside the `<body>` section will be displayed in a browser. The content inside the `<title>` element will be shown in the browser's title bar or in the page's tab.

Tip: You can use either `.htm` or `.html` as file extension. There is no difference; it is up to you.

HTML Basic Examples

HTML Documents

All HTML documents must start with a document type declaration: `<!DOCTYPE html>`.

The HTML document itself begins with `<html>` and ends with `</html>`.

The visible part of the HTML document is between `<body>` and `</body>`.

The `<!DOCTYPE>` Declaration

The `<!DOCTYPE>` declaration represents the document type, and helps browsers to display web pages correctly.

It must only appear once, at the top of the page (before any HTML tags).

The `<!DOCTYPE>` declaration is not case sensitive.

The `<!DOCTYPE>` declaration for HTML5 is:

```
<!DOCTYPE html>
```

HTML Headings

HTML headings are defined with the `<h1>` to `<h6>` tags.

`<h1>` defines the most important heading. `<h6>` defines the least important heading:

Example

`<h1>This is heading 1</h1>`

`<h2>This is heading 2</h2>`

`<h3>This is heading 3</h3>`

HTML Paragraphs

HTML paragraphs are defined with the `<p>` tag:

Example

`<p>This is a paragraph.</p>`

`<p>This is another paragraph.</p>`

HTML Links

HTML links are defined with the `<a>` tag:

Example

`<a href="https://www.w3schools.com">This is a link</a>`

The link's destination is specified in the `href` attribute.

Attributes are used to provide additional information about HTML elements.

You will learn more about attributes in a later chapter.

HTML Images

HTML images are defined with the `<img>` tag.

The source file (`src`), alternative text (`alt`), `width`, and `height` are provided as attributes:

Example

``

How to View HTML Source

Have you ever seen a Web page and wondered "Hey! How did they do that?"

View HTML Source Code:

Click CTRL + U in an HTML page, or right-click on the page and select "View Page Source". This will open a new tab containing the HTML source code of the page.

Inspect an HTML Element:

Right-click on an element (or a blank area), and choose "Inspect" to see what elements are made up of (you will see both the HTML

and the CSS). You can also edit the HTML or CSS on-the-fly in the Elements or Styles panel that opens.

HTML Elements

An HTML element is defined by a start tag, some content, and an end tag.

The HTML **element** is everything from the start tag to the end tag:

`<tagname>Content goes here...</tagname>`

Examples of some HTML elements:

`<h1>My First Heading</h1>`

`<p>My first paragraph.</p>`

Nested HTML Elements

HTML elements can be nested (this means that elements can contain other elements).

All HTML documents consist of nested HTML elements.

The following example contains four HTML elements (`<html>`, `<body>`, `<h1>` and `<p>`):

The `<html>` element is the root element and it defines the whole HTML document.

It has a start tag `<html>` and an end tag `</html>`.

Then, inside the `<html>` element there is a `<body>` element:

The `<body>` element defines the document's body.

It has a start tag `<body>` and an end tag `</body>`.

Then, inside the `<body>` element there are two other elements: `<h1>` and `<p>`:

The `<h1>` element defines a heading.

It has a start tag `<h1>` and an end tag `</h1>`

The `<p>` element defines a paragraph.

It has a start tag `<p>` and an end tag `</p>`

Empty HTML Elements

HTML elements with no content are called empty elements.

The `<br>` tag defines a line break, and is an empty element without a closing tag:

Example

`<p>This is a <br> paragraph with a line break.</p>`

HTML Attributes

- All HTML elements can have **attributes**
- Attributes provide **additional information** about elements
- Attributes are always specified in **the start tag**
- Attributes usually come in name/value pairs like: **name="value"**

The href Attribute

The `<a>` tag defines a hyperlink. The `href` attribute specifies the URL of the page the link goes to:

Example

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

The src Attribute

The `<img>` tag is used to embed an image in an HTML page. The `src` attribute specifies the path to the image to be displayed

```

```

There are two ways to specify the URL in the `src` attribute:

1. Absolute URL - Links to an external image that is hosted on another website.

Example: `src="https://www.w3schools.com/images/img_girl.jpg"`.

Notes: External images might be under copyright. If you do not get permission to use it, you may be in violation of copyright laws. In addition, you cannot control external images; it can suddenly be removed or changed.

2. Relative URL - Links to an image that is hosted within the website. Here, the URL does not include the domain name. If the URL begins without a slash, it will be relative to the current page. Example: `src="img_girl.jpg"`. If the URL begins with a slash, it will be relative to the domain. Example: `src="/images/img_girl.jpg"`.

Tip: It is almost always best to use relative URLs. They will not break if you change domain

The width and height Attributes

The `<img>` tag should also contain the `width` and `height` attributes, which specify the width and height of the image (in pixels):

```

```

The alt Attribute

The required `alt` attribute for the `<img>` tag specifies an alternate text for an image, if the image for some reason cannot be displayed. This can be due to a slow connection, or an error in the `src` attribute, or if the user uses a screen reader.

```

```

```

```

The style Attribute

The `style` attribute is used to add styles to an element, such as color, font, size, and more

```
<p style="color:red;">This is a red paragraph.</p>
```

The lang Attribute

You should always include the `lang` attribute inside the `<html>` tag, to declare the language of the Web page. This is meant to assist search engines and browsers.

The following example specifies English as the language:

```
<!DOCTYPE html>
```

```
<html lang="en">
<body>
...
</body>
</html>
```

The title Attribute

The **title** attribute defines some extra information about an element.

The value of the title attribute will be displayed as a tooltip when you mouse over the element:

```
<p title="I'm a tooltip">This is a paragraph.</p>
```

We Suggest: Always Use Lowercase Attributes

We Suggest: Always Quote Attribute Values

Good:

```
<a href="https://www.w3schools.com/html/">Visit our HTML tutorial</a>
```

Bad:

```
<a href=https://www.w3schools.com/html/>Visit our HTML tutorial</a>
```

Single or Double Quotes?

Double quotes around attribute values are the most common in HTML, but single quotes can also be used.

In some situations, when the attribute value itself contains double quotes, it is necessary to use single quotes:

- All HTML elements can have **attributes**
- The **href** attribute of **<a>** specifies the URL of the page the link goes to
- The **src** attribute of **** specifies the path to the image to be displayed
- The **width** and **height** attributes of **** provide size information for images
- The **alt** attribute of **** provides an alternate text for an image
- The **style** attribute is used to add styles to an element, such as color, font, size, and more
- The **lang** attribute of the **<html>** tag declares the language of the Web page
- The **title** attribute defines some extra information about an element

HTML Headings

HTML headings are titles or subtitles that you want to display on a webpage

HTML headings are defined with the **<h1>** to **<h6>** tags.

<h1> defines the most important heading. **<h6>** defines the least important heading

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
```

<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>

Search engines use the headings to index the structure and content of your web pages

Bigger Headings

Each HTML heading has a default size. However, you can specify the size for any heading with the **style** attribute, using the CSS **font-size** property:

<h1 style="font-size:60px;">Heading 1</h1>

HTML Paragraphs

A paragraph always starts on a new line, and is usually a block of text.

The HTML **<p>** element defines a paragraph.

A paragraph always starts on a new line, and browsers automatically add some white space (a margin) before and after a paragraph.

<p>This is a paragraph.</p>
<p>This is another paragraph.</p>

With HTML, you cannot change the display by adding extra spaces or extra lines in your HTML code. The browser will automatically remove any extra spaces and lines when the page is displayed:

The **<hr>** tag defines a thematic break in an HTML page, and is most often displayed as a horizontal rule. The **<hr>** element is used to separate content (or define a change) in an HTML page:

The **<hr>** tag is an empty tag, which means that it has no end tag.

HTML Line Breaks

The HTML **
** element defines a line break.

Use **
** if you want a line break (a new line) without starting a new paragraph:

<p>This is
a paragraph
with line breaks.</p>

The **
** tag is an empty tag, which means that it has no end tag

The HTML <pre> Element

The HTML **<pre>** element defines preformatted text.

The text inside a **<pre>** element is displayed in a fixed-width font (usually Courier), and it preserves both spaces and line breaks:

Tag	Description
<p>	Defines a paragraph
<hr>	Defines a thematic change in the content

	Inserts a single line break
<pre>	Defines pre-formatted text

HTML Styles

The HTML **style** attribute is used to add styles to an element, such as color, font, size, and more.
The HTML **style** attribute has the following syntax:

```
<tagname style="property:value;">
```

The **property** is a CSS property. The **value** is a CSS value.

Background Color

The CSS **background-color** property defines the background color for an HTML element.
Set the background color for a page to powderblue:

```
<body style="background-color:powderblue;">
```

```
<h1>This is a heading</h1>
```

```
<p>This is a paragraph.</p>
```

```
</body>
```

Text Color

The CSS **color** property defines the text color for an HTML element:

```
<h1 style="color:blue;">This is a heading</h1>
```

```
<p style="color:red;">This is a paragraph.</p>
```

Fonts

The CSS **font-family** property defines the font to be used for an HTML element:

```
<h1 style="font-family:verdana;">This is a heading</h1>
```

```
<p style="font-family:courier;">This is a paragraph.</p>
```

Text Size

The CSS **font-size** property defines the text size for an HTML element:

```
<h1 style="font-size:300%;">This is a heading</h1>
```

```
<p style="font-size:160%;">This is a paragraph.</p>
```

Text Alignment

The CSS **text-align** property defines the horizontal text alignment for an HTML element:

```
<h1 style="text-align:center;">Centered Heading</h1>
```

```
<p style="text-align:center;">Centered paragraph.</p>
```

- Use the **style** attribute for styling HTML elements
- Use **background-color** for background color
- Use **color** for text colors
- Use **font-family** for text fonts
- Use **font-size** for text sizes
- Use **text-align** for text alignment

HTML Text Formatting

HTML contains several elements for defining text with a special meaning.

HTML Formatting Elements

Formatting elements were designed to display special types of text:

- `<b>` - Bold text
- `<strong>` - Important text
- `<i>` - Italic text
- `<em>` - Emphasized text
- `<mark>` - Marked text
- `<small>` - Smaller text
- `<del>` - Deleted text
- `<ins>` - Inserted text
- `<sub>` - Subscript text
- `<sup>` - Superscript text

HTML `<b>` and `<strong>` Elements

The HTML `<b>` element defines bold text, without any extra importance.

`<b>`This text is bold`</b>`

The HTML `<strong>` element defines text with strong importance. The content inside is typically displayed in bold.

`<strong>`This text is important!`</strong>`

The HTML `<i>` element defines a part of text in an alternate voice or mood. The content inside is typically displayed in italic.

`<i>`This text is italic`</i>`

The HTML `<small>` element defines smaller text:

`<small>`This is some smaller text.`</small>`

The HTML `<mark>` element defines text that should be marked or highlighted:

`<p>`Do not forget to buy `<mark>`milk`</mark>` today.`</p>`

The HTML `<del>` element defines text that has been deleted from a document. Browsers will usually strike a line through deleted text:

`<p>`My favorite color is `<del>`blue`</del>` red.`</p>`

The HTML `<ins>` element defines a text that has been inserted into a document. Browsers will usually underline inserted text:

`<p>`My favorite color is `<del>`blue`</del>` `<ins>`red`</ins>`.`</p>`

The HTML `<sub>` element defines subscript text. Subscript text appears half a character below the normal line, and is sometimes rendered in a smaller font. Subscript text can be used for chemical formulas, like H₂O:

`<p>`This is `<sub>`subscripted`</sub>` text.`</p>`

The HTML `<sup>` element defines superscript text. Superscript text appears half a character above the normal line, and is sometimes rendered in a smaller font. Superscript text can be used for footnotes, like WWW^[1]:

`<p>`This is `<sup>`superscripted`</sup>` text.`</p>`

HTML Quotation and Citation Elements

In this chapter we will go through the `<blockquote>`, `<q>`, `<abbr>`, `<address>`, `<cite>`, and `<bdo>` HTML elements

The HTML `<blockquote>` element defines a section that is quoted from another source. Browsers usually indent `<blockquote>` elements.

The HTML `<blockquote>` element defines a section that is quoted from another source. Browsers usually indent `<blockquote>` elements.

The HTML `<blockquote>` element defines a section that is quoted from another source. Browsers usually indent `<blockquote>` elements.

The HTML `<blockquote>` element defines a section that is quoted from another source. Browsers usually indent `<blockquote>` elements.

```
<address>
Written by John Doe.<br>
Visit us at:<br>
Example.com<br>
Box 564, Disneyland<br>
USA
</address>
```

The text in the `<cite>` element usually renders in *italic*.

BDO stands for Bi-Directional Override.

The HTML `<bdo>` tag is used to override the current text direction:

HTML Comment Tag

You can add comments to your HTML source by using the following syntax:

```
<!-- Write your comments here -->
```

```
<!-- This is a comment -->
```

```
<p>This is a paragraph.</p>
```

```
<!-- Remember to add more information here -->
```

Comments can be used to hide content.

This can be helpful if you hide content temporarily:

You can also hide more than one line. Everything between the `<!--` and the `-->` will be hidden from the display.

Comments can be used to hide parts in the middle of the HTML code.

HTML Colors

HTML colors are specified with predefined color names, or with RGB, HEX, HSL, RGBA, or HSLA values.

HTML supports [140 standard color names](#).

Background Color

You can set the background color for HTML elements:

```
<h1 style="background-color:DodgerBlue;">Hello World</h1>
<p style="background-color:Tomato;">Lorem ipsum...</p>
```

Text Color

You can set the color of text:

```
<h1 style="color:Tomato;">Hello World</h1>
<p style="color:DodgerBlue;">Lorem ipsum...</p>
<p style="color:MediumSeaGreen;">Ut wisi enim...</p>
```

Border Color

You can set the color of borders:

```
<h1 style="border:2px solid Tomato;">Hello World</h1>
<h1 style="border:2px solid DodgerBlue;">Hello World</h1>
<h1 style="border:2px solid Violet;">Hello World</h1>
```

Color Values

In HTML, colors can also be specified using RGB values, HEX values, HSL values, RGBA values, and HSLA values. The following three <div> elements have their background color set with RGB, HEX, and HSL values:

```
<h1 style="background-color:rgb(255, 99, 71);">...</h1>
<h1 style="background-color:#ff6347;">...</h1>
<h1 style="background-color:hsl(9, 100%, 64%);">...</h1>

<h1 style="background-color:rgba(255, 99, 71, 0.5);">...</h1>
<h1 style="background-color:hsla(9, 100%, 64%, 0.5);">...</h1>
```

=====

Styling Forms

HTML Styles - CSS

CSS stands for Cascading Style Sheets.

CSS saves a lot of work. It can control the layout of multiple web pages all at once.

Manipulate Text

Colors, Boxes

What is CSS?

Cascading Style Sheets (CSS) is used to format the layout of a webpage.

With CSS, you can control the color, font, the size of text, the spacing between elements, how elements are positioned and laid out, what background images or background colors are to be used, different displays for different devices and screen sizes, and much more!

Using CSS

CSS can be added to HTML documents in 3 ways:

- **Inline** - by using the `style` attribute inside HTML elements
- **Internal** - by using a `<style>` element in the `<head>` section
- **External** - by using a `<link>` element to link to an external CSS file

The most common way to add CSS, is to keep the styles in external CSS files.

Inline CSS

An inline CSS is used to apply a unique style to a single HTML element.

An inline CSS uses the `style` attribute of an HTML element.

The following example sets the text color of the `<h1>` element to blue, and the text color of the `<p>` element to red:

```
<h1 style="color:blue;">A Blue Heading</h1>
<p style="color:red;">A red paragraph.</p>
```

Internal CSS

An internal CSS is used to define a style for a single HTML page.

An internal CSS is defined in the `<head>` section of an HTML page, within a `<style>` element.

The following example sets the text color of ALL the `<h1>` elements (on that page) to blue, and the text color of ALL the `<p>` elements to red. In addition, the page will be displayed with a "powderblue" background color:

```
<!DOCTYPE html>
<html>
<head>
<style>
body {background-color: powderblue;}
h1  {color: blue;}
p   {color: red;}
</style>
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

External CSS

An external style sheet is used to define the style for many HTML pages.

To use an external style sheet, add a link to it in the `<head>` section of each HTML page:

```
<!DOCTYPE html>
<html>
<head>
<link rel="stylesheet" href="styles.css">
</head>
<body>

<h1>This is a heading</h1>
<p>This is a paragraph.</p>

</body>
</html>
```

The external style sheet can be written in any text editor. The file must not contain any HTML code, and must be saved with a .css extension.

Here is what the "styles.css" file looks like:

"styles.css":

```
body {  
  
    background-color: powderblue;  
  
}  
  
h1 {  
  
    color: blue;  
  
}  
  
p {  
  
    color: red;  
  
}
```

Tip: With an external style sheet, you can change the look of an entire web site, by changing one file!

CSS Colors, Fonts and Sizes

Here, we will demonstrate some commonly used CSS properties. You will learn more about them later.

The CSS **color** property defines the text color to be used.

The CSS **font-family** property defines the font to be used.

The CSS **font-size** property defines the text size to be used.

Example

Use of CSS color, font-family and font-size properties:

```
<!DOCTYPE html>  
  
<html>  
  
<head>  
  
<style>  
  
h1 {  
  
    color: blue;  
  
    font-family: verdana;  
  
    font-size: 300%;  
  
}  
  
p {  
  
    color: red;  
  
    font-family: courier;
```

```

font-size: 160%;
}

</style>

</head>

<body>

<h1>This is a heading</h1>

<p>This is a paragraph.</p>

</body>

</html>

```

CSS Border

The CSS **border** property defines a border around an HTML element.

```

p {
    border: 2px solid powderblue;
}

```

CSS Padding

The CSS **padding** property defines a padding (space) between the text and the border.

```

p {
    border: 2px solid powderblue;
    padding: 30px;
}

```

The CSS **margin** property defines a margin (space) outside the border.

```

p {
    border: 2px solid powderblue;
    margin: 50px;
}

```

Link to External CSS

External style sheets can be referenced with a full URL or with a path relative to the current web page. This example uses a full URL to link to a style sheet:

```
<link rel="stylesheet" href="https://www.w3schools.com/html/styles.css">
```

This example links to a style sheet located in the html folder on the current web site:

```
<link rel="stylesheet" href="/html/styles.css">
```

This example links to a style sheet located in the same folder as the current page:

```
<link rel="stylesheet" href="styles.css">
```

- Use the HTML **style** attribute for inline styling
- Use the HTML **<style>** element to define internal CSS
- Use the HTML **<link>** element to refer to an external CSS file
- Use the HTML **<head>** element to store **<style>** and **<link>** elements
- Use the CSS **color** property for text colors
- Use the CSS **font-family** property for text fonts
- Use the CSS **font-size** property for text sizes
- Use the CSS **border** property for borders
- Use the CSS **padding** property for space inside the border
- Use the CSS **margin** property for space outside the border

HTML Links

Links are found in nearly all web pages. Links allow users to click their way from page to page.

HTML links are hyperlinks.

You can click on a link and jump to another document.

When you move the mouse over a link, the mouse arrow will turn into a little hand.

Note: A link does not have to be text. A link can be an image or any other HTML element!

The HTML **<a>** tag defines a hyperlink. It has the following syntax:

```
<a href="url">Link text</a>
```

The most important attribute of the **<a>** element is the **href** attribute, which indicates the link's destination.

The *link text* is the part that will be visible to the reader.

Clicking on the link text, will send the reader to the specified URL address.

This example shows how to create a link to W3Schools.com:

```
<a href="https://www.w3schools.com/">Visit W3Schools.com!</a>
```

By default, links will appear as follows in all browsers:

- An unvisited link is underlined and blue
- A visited link is underlined and purple
- An active link is underlined and red

HTML Links - The target Attribute

By default, the linked page will be displayed in the current browser window. To change this, you must specify another target for the link.

The **target** attribute specifies where to open the linked document.

The **target** attribute can have one of the following values:

- **_self** - Default. Opens the document in the same window/tab as it was clicked
- **_blank** - Opens the document in a new window or tab
- **_parent** - Opens the document in the parent frame
- **_top** - Opens the document in the full body of the window

Use **target="_blank"** to open the linked document in a new browser window or tab:

```
<a href="https://www.w3schools.com/" target="_blank">Visit W3Schools!</a>
```

Absolute URLs vs. Relative URLs

Both examples above are using an **absolute URL** (a full web address) in the **href** attribute.

A local link (a link to a page within the same website) is specified with a **relative URL** (without the "<https://www>" part):

```
<h2>Absolute URLs</h2>
<p><a href="https://www.w3.org/">W3C</a></p>
<p><a href="https://www.google.com/">Google</a></p>

<h2>Relative URLs</h2>
<p><a href="html_images.asp">HTML Images</a></p>
<p><a href="/css/default.asp">CSS Tutorial</a></p>
```

HTML Links - Use an Image as a Link

To use an image as a link, just put the `<img>` tag inside the `<a>` tag:

Example

```
<a href="default.asp">

</a>
```

Link to an Email Address

Use **mailto:** inside the **href** attribute to create a link that opens the user's email program (to let them send a new email):

```
<a href="mailto:someone@example.com">Send email</a>
```

Button as a Link

To use an HTML button as a link, you have to add some JavaScript code.

JavaScript allows you to specify what happens at certain events, such as a click of a button:

```
<button onclick="document.location='default.asp'">HTML Tutorial</button>
```

Link Titles

The **title** attribute specifies extra information about an element. The information is most often shown as a tooltip text when the mouse moves over the element.

```
<a href="https://www.w3schools.com/html/" title="Go to W3Schools HTML section">Visit our HTML Tutorial</a>
```

More on Absolute URLs and Relative URLs

Use a full URL to link to a web page:

```
<a href="https://www.w3schools.com/html/default.asp">HTML tutorial</a>
```

Link to a page located in the html folder on the current web site:

```
<a href="/html/default.asp">HTML tutorial</a>
```

Link to a page located in the same folder as the current page:

```
<a href="default.asp">HTML tutorial</a>
```

- Use the `<a>` element to define a link
- Use the `href` attribute to define the link address
- Use the `target` attribute to define where to open the linked document
- Use the `<img>` element (inside `<a>`) to use an image as a link
- Use the `mailto:` scheme inside the `href` attribute to create a link that opens the user's email program

HTML Link Colors

By default, a link will appear like this (in all browsers):

- An unvisited link is underlined and blue
- A visited link is underlined and purple
- An active link is underlined and red

You can change the link state colors, by using CSS:

Link Buttons

A link can also be styled as a button, by using CSS:

HTML links can be used to create bookmarks, so that readers can jump to specific parts of a web page.

Create a Bookmark in HTML

Bookmarks can be useful if a web page is very long.

To create a bookmark - first create the bookmark, then add a link to it.

When the link is clicked, the page will scroll down or up to the location with the bookmark

First, use the `id` attribute to create a bookmark:

```
<h2 id="C4">Chapter 4</h2>
```

Then, add a link to the bookmark ("Jump to Chapter 4"), from within the same page:

```
<a href="#C4">Jump to Chapter 4</a>
```

You can also add a link to a bookmark on another page:

```
<a href="html_demo.html#C4">Jump to Chapter 4</a>
```

HTML Images

Images can improve the design and the appearance of a web page.

Example

```

```

```


```

HTML Images Syntax

The HTML `<img>` tag is used to embed an image in a web page.

Images are not technically inserted into a web page; images are linked to web pages. The `<img>` tag creates a holding space for the referenced image.

The `<img>` tag is empty, it contains attributes only, and does not have a closing tag.

The `<img>` tag has two required attributes:

- `src` - Specifies the path to the image
- `alt` - Specifies an alternate text for the image

```

```

The src Attribute

The required `src` attribute specifies the path (URL) to the image.

```

```

The alt Attribute

The required `alt` attribute provides an alternate text for an image, if the user for some reason cannot view it (because of slow connection, an error in the `src` attribute, or if the user uses a screen reader).

The value of the `alt` attribute should describe the image:

```

```

If a browser cannot find an image, it will display the value of the `alt` attribute:

```

```

Image Size - Width and Height

You can use the `style` attribute to specify the width and height of an image.

```

```

Alternatively, you can use the `width` and `height` attributes:

```

```

The `width` and `height` attributes always define the width and height of the image in pixels.

Width and Height, or Style?

The `width`, `height`, and `style` attributes are all valid in HTML.

However, we suggest using the `style` attribute. It prevents styles sheets from changing the size of images:

Images in Another Folder

If you have your images in a sub-folder, you must include the folder name in the `src` attribute:

```

```

Images on Another Server/Website

Some web sites point to an image on another server.

To point to an image on another server, you must specify an absolute (full) URL in the **src** attribute:

```

```

Animated Images

HTML allows animated GIFs:

```

```

Image as a Link

To use an image as a link, put the **** tag inside the **<a>** tag:

```
<a href="default.asp">
  
</a>
```

Image Floating

Use the CSS **float** property to let the image float to the right or to the left of a text:

```
<p>
The image will float to the right of the text.</p>
```

```
<p>
The image will float to the left of the text.</p>
```

- Use the HTML **** element to define an image
- Use the HTML **src** attribute to define the URL of the image
- Use the HTML **alt** attribute to define an alternate text for an image, if it cannot be displayed
- Use the HTML **width** and **height** attributes or the CSS **width** and **height** properties to define the size of the image
- Use the CSS **float** property to let the image float to the left or to the right

=====

HTML Image Maps

With HTML image maps, you can create clickable areas on an image.

Image Maps

The HTML **<map>** tag defines an image map. An image map is an image with clickable areas. The areas are defined with one or more **<area>** tags.

Here is the HTML source code for the image map above:

```

```

```
<map name="workmap">
```

```

<area shape="rect" coords="34,44,270,350" alt="Computer" href="computer.htm">

<area shape="rect" coords="290,172,333,250" alt="Phone" href="phone.htm">

<area shape="circle" coords="337,300,44" alt="Coffee" href="coffee.htm">

</map>

```

- Use the HTML `<map>` element to define an image map
- Use the HTML `<area>` element to define the clickable areas in the image map
- Use the HTML `usemap` attribute of the `<img>` element to point to an image map

From https://www.w3schools.com/html/html_images_imagemap.asp

HTML Background Images

A background image can be specified for almost any HTML element.

Background Image on an HTML element

To add a background image on an HTML element, use the HTML `style` attribute and the CSS `background-image` property:

Add a background image on a `<p>` element:

```

<p style="background-image: url('img_girl.jpg');">

```

You can also specify the background image in the `<style>` element, in the `<head>` section:

Specify the background image in the `<style>` element:

```

<style>
p {
  background-image: url('img_girl.jpg');
}
</style>

```

Background Image on a Page

If you want the entire page to have a background image, you must specify the background image on the `<body>` element:

```

<style>
body {
  background-image: url('img_girl.jpg');
}
</style>

```

Background Repeat

If the background image is smaller than the element, the image will repeat itself, horizontally and vertically, until it reaches the end of the element:

```

<style>
body {
  background-image: url('example_img_girl.jpg');
}
</style>

```

To avoid the background image from repeating itself, set the **background-repeat** property to **no-repeat**

```
<style>
body {
  background-image: url('example_img_girl.jpg');
  background-repeat: no-repeat;
}
</style>
```

If you want the background image to cover the entire element, you can set the **background-size** property to **cover**. Also, to make sure the entire element is always covered, set the **background-attachment** property to **fixed**. This way, the background image will cover the entire element, with no stretching (the image will keep its original proportions):

```
<style>
body {
  background-image: url('img_girl.jpg');
  background-repeat: no-repeat;
  background-attachment: fixed;
  background-size: cover;
}
</style>
```

The HTML **<picture>** element allows you to display different pictures for different devices or screen sizes.

The HTML **<picture>** element gives web developers more flexibility in specifying image resources. The **<picture>** element contains one or more **<source>** elements, each referring to different images through the **srcset** attribute. This way the browser can choose the image that best fits the current view and/or device. Each **<source>** element has a **media** attribute that defines when the image is the most suitable.

Show different images for different screen sizes:

```
<picture>
  <source media="(min-width: 650px)" srcset="img_food.jpg">
  <source media="(min-width: 465px)" srcset="img_car.jpg">
  
</picture>
```

HTML Lists

HTML lists allow web developers to group a set of related items in lists.

1. Unordered list
2. Ordered list
3. Other list

An unordered HTML list:

- Item
- Item
- Item
- Item

An ordered HTML list:

1. First item
2. Second item
3. Third item
4. Fourth item

Unordered HTML List

An unordered list starts with the `<ul>` tag. Each list item starts with the `<li>` tag. The list items will be marked with bullets (small black circles) by default:

```
<ul>
<li>Coffee</li>
<li>Tea</li>
<li>Milk</li>
</ul>
```

Ordered HTML List

An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag. The list items will be marked with numbers by default:

```
<ol>
<li>Coffee</li>
<li>Tea</li>
<li>Milk</li>
</ol>
```

From https://www.w3schools.com/html/html_lists.asp

HTML Description Lists

HTML also supports description lists.

A description list is a list of terms, with a description of each term.

The `<dl>` tag defines the description list, the `<dt>` tag defines the term (name), and the `<dd>` tag describes each term:

```
<dl>
<dt>Coffee</dt>
<dd>- black hot drink</dd>
<dt>Milk</dt>
<dd>- white cold drink</dd>
</dl>
```

Unordered HTML List - Choose List Item Marker

The CSS `list-style-type` property is used to define the style of the list item marker. It can have one of the following values:

Value	Description
disc	Sets the list item marker to a bullet (default)
circle	Sets the list item marker to a circle
square	Sets the list item marker to a square
none	The list items will not be marked

```
<ul style="list-style-type:disc;">
<li>Coffee</li>
<li>Tea</li>
<li>Milk</li>
</ul>

<ul style="list-style-type:circle;">
<li>Coffee</li>
<li>Tea</li>
<li>Milk</li>
</ul>
```

```

<ul style="list-style-type:square;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>

<ul style="list-style-type:none;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>

```

No List Marker

```

<ul style="list-style-type:none;">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ul>

```

Nested HTML Lists

Lists can be nested (list inside list):

```

<ul>
  <li>Coffee</li>
  <li>Tea
    <ul>
      <li>Black tea</li>
      <li>Green tea</li>
    </ul>
  </li>
  <li>Milk</li>
</ul>

```

Horizontal List with CSS

HTML lists can be styled in many different ways with CSS.

One popular way is to style a list horizontally, to create a navigation menu:

Example:

```

<!DOCTYPE html>
<html>
<head>
<style>
ul {
  list-style-type: none;
  margin: 0;
  padding: 0;
  overflow: hidden;
  background-color: #333333;
}

li {
  float: left;
}

li a {
  display: block;
  color: white;
  text-align: center;
  padding: 16px;
  text-decoration: none;
}

```

```

}

li a:hover {
    background-color: #111111;
}
</style>
</head>
<body>

<ul>
<li><a href="#home">Home</a></li>
<li><a href="#news">News</a></li>
<li><a href="#contact">Contact</a></li>
<li><a href="#about">About</a></li>
</ul>

</body>
</html>

```

- Use the HTML `<ul>` element to define an unordered list
- Use the CSS `list-style-type` property to define the list item marker
- Use the HTML `<li>` element to define a list item
- Lists can be nested
- List items can contain other HTML elements
- Use the CSS property `float:left` to display a list horizontally

From https://www.w3schools.com/html/html_lists_unordered.asp

HTML Ordered Lists

The HTML `<ol>` tag defines an ordered list. An ordered list can be numerical or alphabetical.

Ordered HTML List

An ordered list starts with the `<ol>` tag. Each list item starts with the `<li>` tag. The list items will be marked with numbers by default:

```

<ol>
<li>Coffee</li>
<li>Tea</li>
<li>Milk</li>
</ol>

```

Ordered HTML List - The Type Attribute

The `type` attribute of the `<ol>` tag, defines the type of the list item marker:

Type	Description
type="1"	The list items will be numbered with numbers (default)
type="A"	The list items will be numbered with uppercase letters
type="a"	The list items will be numbered with lowercase letters
type="I"	The list items will be numbered with uppercase roman numbers
type="i"	The list items will be numbered with lowercase roman numbers

```

<ol type="1">
<li>Coffee</li>
<li>Tea</li>
<li>Milk</li>

```

```

</ol>

<ol type="A">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>

<ol type="a">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>

<ol type="I">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>

<ol type="i">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>

```

Control List Counting

By default, an ordered list will start counting from 1. If you want to start counting from a specified number, you can use the **start** attribute:

```

<ol start="50">
  <li>Coffee</li>
  <li>Tea</li>
  <li>Milk</li>
</ol>

```

Nested HTML Lists

Lists can be nested (list inside list):

```

<ol>
  <li>Coffee</li>
  <li>Tea
    <ol>
      <li>Black tea</li>
      <li>Green tea</li>
    </ol>
  </li>
  <li>Milk</li>
</ol>

```

- Use the HTML **** element to define an ordered list
- Use the HTML **type** attribute to define the numbering type
- Use the HTML **** element to define a list item
- Lists can be nested
- List items can contain other HTML elements

HTML List Tags

Tag	Description
<u></u>	Defines an unordered list

<code></code>	Defines an ordered list
<code></code>	Defines a list item
<code><dl></code>	Defines a description list
<code><dt></code>	Defines a term in a description list
<code><dd></code>	Describes the term in a description list

From https://www.w3schools.com/html/html_lists_ordered.asp

HTML Description Lists

A description list is a list of terms, with a description of each term.

The [`<dl>`](#) tag defines the description list, the [`<dt>`](#) tag defines the term (name), and the [`<dd>`](#) tag describes each term:

```
<dl>
  <dt>Coffee</dt>
  <dd>- black hot drink</dd>
  <dt>Milk</dt>
  <dd>- white cold drink</dd>
</dl>
```

- Use the HTML `<dl>` element to define a description list
- Use the HTML `<dt>` element to define the description term
- Use the HTML `<dd>` element to describe the term in a description list

=====

HTML Tables

HTML tables allow web developers to arrange data into rows and columns.

A table in HTML consists of table cells inside rows and columns.

```
<table>
<tr>
  <th>Company</th>
  <th>Contact</th>
  <th>Country</th>
</tr>
<tr>
  <td>Alfreds Futterkiste</td>
  <td>Maria Anders</td>
  <td>Germany</td>
</tr>
<tr>
  <td>Centro comercial Moctezuma</td>
  <td>Francisco Chang</td>
  <td>Mexico</td>
</tr>
</table>
```

Table Cells

Each table cell is defined by a `<td>` and a `</td>` tag.

`td` stands for table data.

Everything between `<td>` and `</td>` is the content of a table cell.

```
<table>
```

```

<tr>
  <td>Emil</td>
  <td>Tobias</td>
  <td>Linus</td>
</tr>
</table>

```

Note: A table cell can contain all sorts of HTML elements: text, images, lists, links, other tables, etc.

Table Rows

Each table row starts with a `<tr>` and ends with a `</tr>` tag.
`tr` stands for table row.

```

<table>
  <tr>
    <td>Emil</td>
    <td>Tobias</td>
    <td>Linus</td>
  </tr>
  <tr>
    <td>16</td>
    <td>14</td>
    <td>10</td>
  </tr>
</table>

```

You can have as many rows as you like in a table; just make sure that the number of cells are the same in each row.

Table Headers

Sometimes you want your cells to be table header cells. In those cases use the `<th>` tag instead of the `<td>` tag:
`th` stands for table header.

```

<table>
  <tr>
    <th>Person 1</th>
    <th>Person 2</th>
    <th>Person 3</th>
  </tr>
  <tr>
    <td>Emil</td>
    <td>Tobias</td>
    <td>Linus</td>
  </tr>
  <tr>
    <td>16</td>
    <td>14</td>
    <td>10</td>
  </tr>
</table>

```

By default, the text in `<th>` elements are bold and centered, but you can change that with CSS.

HTML Table Tags

Tag	Description
<code><table></code>	Defines a table
<code><th></code>	Defines a header cell in a table
<code><tr></code>	Defines a row in a table

<code><td></code>	Defines a cell in a table
<code><caption></code>	Defines a table caption
<code><colgroup></code>	Specifies a group of one or more columns in a table for formatting
<code><col></code>	Specifies column properties for each column within a <code><colgroup></code> element
<code><thead></code>	Groups the header content in a table
<code><tbody></code>	Groups the body content in a table
<code><tfoot></code>	Groups the footer content in a table

HTML Table Borders

HTML tables can have borders of different styles and shapes.

How To Add a Border

To add a border, use the CSS `border` property on `table`, `th`, and `td` elements:

```
table, th, td {
  border: 1px solid black;
}
```

Collapsed Table Borders

To avoid having double borders like in the example above, set the CSS `border-collapse` property to `collapse`. This will make the borders collapse into a single border:

```
table, th, td {
  border: 1px solid black;
  border-collapse: collapse;
}
```

Style Table Borders

If you set a background color of each cell, and give the border a white color (the same as the document background), you get the impression of an invisible border:

```
table, th, td {
  border: 1px solid white;
  border-collapse: collapse;
}
th, td {
  background-color: #96D4D4;
}
```

Round Table Borders

With the `border-radius` property, the borders get rounded corners:

```
table, th, td {
  border: 1px solid black;
  border-radius: 10px;
}
```

Skip the border around the table by leaving out `table` from the css selector:

```
th, td {
  border: 1px solid black;
  border-radius: 10px;
}
```

Dotted Table Borders

With the `border-style` property, you can set the appearance of the border.

The following values are allowed:

- `dotted`
- `dashed`
- `solid`
- `double`
- `groove`
- `ridge`
- `inset`
- `outset`
- `none`
- `hidden`

```
th, td {
  border-style: dotted;
}
```

Border Color

With the `border-color` property, you can set the color of the border.

```
th, td {
  border-color: #96D4D4;
}
```

HTML Table Sizes

HTML tables can have different sizes for each column, row or the entire table.

Use the `style` attribute with the `width` or `height` properties to specify the size of a table, row or column.

HTML Table Width

To set the width of a table, add the `style` attribute to the `<table>` element:

```
<table style="width:100%">
<tr>
  <th>Firstname</th>
  <th>Lastname</th>
  <th>Age</th>
</tr>
<tr>
  <td>Jill</td>
  <td>Smith</td>
  <td>50</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
```

```
</table>
```

HTML Table Column Width

To set the size of a specific column, add the **style** attribute on a **<th>** or **<td>** element:
Set the width of the first column to 70%:

```
<table style="width:100%">
<tr>
  <th style="width:70%">Firstname</th>
  <th>Lastname</th>
  <th>Age</th>
</tr>
<tr>
  <td>Jill</td>
  <td>Smith</td>
  <td>50</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
</table>
```

HTML Table Row Height

To set the height of a specific row, add the **style** attribute on a table row element:
Set the height of the second row to 200 pixels:

```
<table style="width:100%">
<tr>
  <th>Firstname</th>
  <th>Lastname</th>
  <th>Age</th>
</tr>
<tr style="height:200px">
  <td>Jill</td>
  <td>Smith</td>
  <td>50</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
</table>
```

HTML Table Headers

HTML tables can have headers for each column or row, or for many columns/rows.

Table headers are defined with **th** elements. Each **th** element represents a table cell.

```
<table>
<tr>
  <th>Firstname</th>
  <th>Lastname</th>
  <th>Age</th>
</tr>
```

```

<tr>
  <td>Jill</td>
  <td>Smith</td>
  <td>50</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
</table>

```

Vertical Table Headers

To use the first column as table headers, define the first cell in each row as a `<th>` element:

```

<table>
  <tr>
    <th>Firstname</th>
    <td>Jill</td>
    <td>Eve</td>
  </tr>
  <tr>
    <th>Lastname</th>
    <td>Smith</td>
    <td>Jackson</td>
  </tr>
  <tr>
    <th>Age</th>
    <td>94</td>
    <td>50</td>
  </tr>
</table>

```

Align Table Headers

By default, table headers are bold and centered:

Firstname	Lastname	Age
Jill	Smith	50
Eve	Jackson	94

To left-align the table headers, use the CSS `text-align` property:

```

th {
  text-align: left;
}

```

Header for Multiple Columns

You can have a header that spans over two or more columns.

Name		Age
Jill	Smith	50
Eve	Jackson	94

To do this, use the `colspan` attribute on the `<th>` element:

```

<table>
  <tr>
    <th colspan="2">Name</th>
    <th>Age</th>
  </tr>

```

```

<tr>
  <td>Jill</td>
  <td>Smith</td>
  <td>50</td>
</tr>
<tr>
  <td>Eve</td>
  <td>Jackson</td>
  <td>94</td>
</tr>
</table>

```

Table Caption

You can add a caption that serves as a heading for the entire table.

Month	Savings
January	\$100
February	\$50

Monthly savings

To add a caption to a table, use the `<caption>` tag:

```

<table style="width:100%">
  <caption>Monthly savings</caption>
  <tr>
    <th>Month</th>
    <th>Savings</th>
  </tr>
  <tr>
    <td>January</td>
    <td>$100</td>
  </tr>
  <tr>
    <td>February</td>
    <td>$50</td>
  </tr>
</table>

```

HTML Table Padding & Spacing

HTML tables can adjust the padding inside the cells, and also the space between the cells.

HTML Table - Cell Padding

Cell padding is the space between the cell edges and the cell content.

By default the padding is set to 0.

To add padding on table cells, use the CSS `padding` property:

```

th, td {
  padding: 15px;
}

```

To add padding only above the content, use the `padding-top` property.

Add padding to the other sides with the `padding-bottom`, `padding-left`, and `padding-right` properties:

```

th, td {
  padding-top: 10px;
  padding-bottom: 20px;
  padding-left: 30px;
  padding-right: 40px;
}

```

```
}
```

HTML Table - Cell Spacing

Cell spacing is the space between each cell.

By default the space is set to 2 pixels.

To change the space between table cells, use the CSS **border-spacing** property on the **table** element:

```
table {  
    border-spacing: 30px;  
}
```

HTML Table Colspan & Rowspan

HTML tables can have cells that span over multiple rows and/or columns.

HTML Table - Colspan

To make a cell span over multiple columns, use the **colspan** attribute:

```
<table>  
<tr>  
  <th colspan="2">Name</th>  
  <th>Age</th>  
</tr>  
<tr>  
  <td>Jill</td>  
  <td>Smith</td>  
  <td>43</td>  
</tr>  
<tr>  
  <td>Eve</td>  
  <td>Jackson</td>  
  <td>57</td>  
</tr>  
</table>
```

Note: The value of the **colspan** attribute represents the number of columns to span.

HTML Table - Rowspan

To make a cell span over multiple rows, use the **rowspan** attribute:

```
<table>  
<tr>  
  <th>Name</th>  
  <td>Jill</td>  
</tr>  
<tr>  
  <th rowspan="2">Phone</th>  
  <td>555-1234</td>  
</tr>  
<tr>  
  <td>555-8745</td>  
</tr>  
</table>
```

Note: The value of the **rowspan** attribute represents the number of rows to span.

HTML Table Styling

Use CSS to make your tables look better.

HTML Table - Zebra Stripes

If you add a background color on every other table row, you will get a nice zebra stripes effect. To style every other table row element, use the `:nth-child(even)` selector like this:

```
tr:nth-child(even) {  
    background-color: #D6EEEE;  
}
```

Note: If you use `(odd)` instead of `(even)`, the styling will occur on row 1,3,5 etc. instead of 2,4,6 etc.

HTML Table - Vertical Zebra Stripes

To make vertical zebra stripes, style every other *column*, instead of every other *row*. Set the `:nth-child(even)` for table data elements like this:

```
td:nth-child(even), th:nth-child(even) {  
    background-color: #D6EEEE;  
}
```

Note: Put the `:nth-child()` selector on both `th` and `td` elements if you want to have the styling on both headers and regular table cells.

Combine Vertical and Horizontal Zebra Stripes

You can combine the styling from the two examples above and you will have stripes on every other row and every other column. If you use a transparent color you will get an overlapping effect. Use an `rgba()` color to specify the transparency of the color:

```
tr:nth-child(even) {  
    background-color: rgba(150, 212, 212, 0.4);  
}  
  
th:nth-child(even),td:nth-child(even) {  
    background-color: rgba(150, 212, 212, 0.4);  
}
```

Horizontal Dividers

If you specify borders only at the bottom of each table row, you will have a table with horizontal dividers. Add the `border-bottom` property to all `tr` elements to get horizontal dividers:

```
tr {  
    border-bottom: 1px solid #ddd;  
}
```

Hoverable Table

Use the `:hover` selector on `tr` to highlight table rows on mouse over:

```
tr:hover {background-color: #D6EEEE;}
```

HTML Table Colgroup

The `<colgroup>` element is used to style specific columns of a table.

HTML Table Colgroup

If you want to style the first two columns of a table, use the `<colgroup>` and `<col>` elements.

The `<colgroup>` element should be used as a container for the column specifications.

Each group is specified with a `<col>` element.

The `span` attribute specifies how many columns get the style.

The `style` attribute specifies the style to give the columns.

```
<table>
<colgroup>
  <col span="2" style="background-color: #D6EEEE">
</colgroup>
<tr>
  <th>MON</th>
  <th>TUE</th>
  <th>WED</th>
  <th>THU</th>
```

Note: The `<colgroup>` tag must be a child of a `<table>` element and should be placed before any other table elements, like `<thead>`, `<tr>`, `<td>` etc., but after the `<caption>` element, if present.

Multiple Col Elements

If you want to style multiple columns with different styles, use more than one `<col>` element inside the `<colgroup>`:

```
<table>
<colgroup>
  <col span="2" style="background-color: #D6EEEE">
  <col span="3" style="background-color: pink">
</colgroup>
<tr>
  <th>MON</th>
  <th>TUE</th>
  <th>WED</th>
  <th>THU</th>
```

Empty Colgroups

If you want to style columns in the middle of a table, insert an "empty" `<col>` element (with no styles) for the columns before:

```
<table>
<colgroup>
  <col span="3">
  <col span="2" style="background-color: pink">
</colgroup>
<tr>
  <th>MON</th>
  <th>TUE</th>
  <th>WED</th>
  <th>THU</th>
```

Hide Columns

You can hide columns with the **visibility: collapse** property:

```
<table>
<colgroup>
  <col span="2">
  <col span="3" style="visibility: collapse">
</colgroup>
<tr>
  <th>MON</th>
  <th>TUE</th>
  <th>WED</th>
  <th>THU</th>
```

HTML Forms

An HTML form is used to collect user input. The user input is most often sent to a server for processing.

First name:

Last name:

The HTML **<form>** element is used to create an HTML form for user input:

```
<form>
```

.

form elements

.

```
</form>
```

The **<form>** element is a container for different types of input elements, such as: text fields, checkboxes, radio buttons, submit buttons, etc.

The <input> Element

The HTML **<input>** element is the most used form element.

An **<input>** element can be displayed in many ways, depending on the **type** attribute.

Here are some examples:

Type	Description
<input type="text">	Displays a single-line text input field
<input type="radio">	Displays a radio button (for selecting one of many choices)
<input type="checkbox">	Displays a checkbox (for selecting zero or more of many choices)
<input type="submit">	Displays a submit button (for submitting the form)
<input type="button">	Displays a clickable button

Text Fields

The **<input type="text">** defines a single-line input field for text input.

```
<form>
```

```

<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname">
</form>

```

The <label> Element

Notice the use of the <label> element in the example above.

The <label> tag defines a label for many form elements.

The <label> element is useful for screen-reader users, because the screen-reader will read out loud the label when the user focuses on the input element.

The <label> element also helps users who have difficulty clicking on very small regions (such as radio buttons or checkboxes) - because when the user clicks the text within the <label> element, it toggles the radio button/checkbox.

The **for** attribute of the <label> tag should be equal to the **id** attribute of the <input> element to bind them together.

Radio Buttons

The <input type="radio"> defines a radio button.

Radio buttons let a user select ONE of a limited number of choices.

```

<p>Choose your favorite Web language:</p>

```

```

<form>

<input type="radio" id="html" name="fav_language" value="HTML">

<label for="html">HTML</label><br>

<input type="radio" id="css" name="fav_language" value="CSS">

<label for="css">CSS</label><br>

<input type="radio" id="javascript" name="fav_language" value="JavaScript">

<label for="javascript">JavaScript</label>

</form>

```

This is how the HTML code above will be displayed in a browser:

Choose your favorite Web language:

HTML

CSS

JavaScript

Checkboxes

The <input type="checkbox"> defines a **checkbox**.

Checkboxes let a user select ZERO or MORE options of a limited number of choices.

```

<form>
<input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
<label for="vehicle1"> I have a bike</label><br>
<input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
<label for="vehicle2"> I have a car</label><br>

```

```
<input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
<label for="vehicle3"> I have a boat</label>
</form>
```

This is how the HTML code above will be displayed in a browser:

I have a bike
I have a car
I have a boat

The Submit Button

The `<input type="submit">` defines a button for submitting the form data to a form-handler.

The form-handler is typically a file on the server with a script for processing input data.

The form-handler is specified in the form's `action` attribute.

```
<form action="/action_page.php">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
</form>
```

The Name Attribute for <input>

Notice that each input field must have a `name` attribute to be submitted.

If the `name` attribute is omitted, the value of the input field will not be sent at all.

```
<form action="/action_page.php">
<label for="fname">First name:</label><br>
<input type="text" id="fname" value="John"><br><br>
<input type="submit" value="Submit">
</form>
```

HTML Form Attributes

The Action Attribute

The `action` attribute defines the action to be performed when the form is submitted.

Usually, the form data is sent to a file on the server when the user clicks on the submit button.

In the example below, the form data is sent to a file called "action_page.php". This file contains a server-side script that handles the form data:

On submit, send form data to "action_page.php":

```
<form action="/action_page.php">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
</form>
```

Tip: If the `action` attribute is omitted, the action is set to the current page.

The Target Attribute

The **target** attribute specifies where to display the response that is received after submitting the form. The **target** attribute can have one of the following values:

Value	Description
_blank	The response is displayed in a new window or tab
_self	The response is displayed in the current window
_parent	The response is displayed in the parent frame
_top	The response is displayed in the full body of the window
framename	The response is displayed in a named iframe

The default value is **_self** which means that the response will open in the current window.

Here, the submitted result will open in a new browser tab:

```
<form action="/action_page.php" target="_blank">
```

The Method Attribute

The **method** attribute specifies the HTTP method to be used when submitting the form data. The form-data can be sent as URL variables (with **method="get"**) or as HTTP post transaction (with **method="post"**). The default HTTP method when submitting form data is GET.

This example uses the GET method when submitting the form data:

```
<form action="/action_page.php" method="get">
```

This example uses the POST method when submitting the form data:

```
<form action="/action_page.php" method="post">
```

Notes on GET:

- Appends the form data to the URL, in name/value pairs
- NEVER use GET to send sensitive data! (the submitted form data is visible in the URL!)
- The length of a URL is limited (2048 characters)
- Useful for form submissions where a user wants to bookmark the result
- GET is good for non-secure data, like query strings in Google

Notes on POST:

- Appends the form data inside the body of the HTTP request (the submitted form data is not shown in the URL)
- POST has no size limitations, and can be used to send large amounts of data.
- Form submissions with POST cannot be bookmarked

The Autocomplete Attribute

The **autocomplete** attribute specifies whether a form should have autocomplete on or off. When autocomplete is on, the browser automatically complete values based on values that the user has entered before.

A form with autocomplete on:

```
<form action="/action_page.php" autocomplete="on">
```

The Novalidate Attribute

The **novalidate** attribute is a boolean attribute. When present, it specifies that the form-data (input) should not be validated when submitted.

A form with a novalidate attribute:

```
<form action="/action_page.php" novalidate>
```

List of All <form> Attributes

Attribute	Description
accept-charset	Specifies the character encodings used for form submission
action	Specifies where to send the form-data when a form is submitted
autocomplete	Specifies whether a form should have autocomplete on or off
enctype	Specifies how the form-data should be encoded when submitting it to the server (only for method="post")
method	Specifies the HTTP method to use when sending form-data
name	Specifies the name of the form
novalidate	Specifies that the form should not be validated when submitted
rel	Specifies the relationship between a linked resource and the current document
target	Specifies where to display the response that is received after submitting the form

HTML Form Elements

This chapter describes all the different HTML form elements.

The HTML <form> element can contain one or more of the following form elements:

- <input>
- <label>
- <select>
- <textarea>
- <button>
- <fieldset>
- <legend>
- <datalist>
- <output>
- <option>
- <optgroup>

The <input> Element

One of the most used form elements is the <input> element.

The <input> element can be displayed in several ways, depending on the type attribute.

Example

```
<label for="fname">First name:</label>
```

```
<input type="text" id="fname" name="fname">
```

The <label> Element

The <label> element defines a label for several form elements.

The <label> element is useful for screen-reader users, because the screen-reader will read out loud the label when the user focus on the input element.

The <label> element also help users who have difficulty clicking on very small regions (such as radio buttons or checkboxes) - because when the user clicks the text within the <label> element, it toggles the radio button/checkbox.

The for attribute of the <label> tag should be equal to the id attribute of the <input> element to bind them together.

The <select> Element

The <select> element defines a drop-down list:

```
<label for="cars">Choose a car:</label>
<select id="cars" name="cars">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat">Fiat</option>
  <option value="audi">Audi</option>
</select>
```

The <option> element defines an option that can be selected.

By default, the first item in the drop-down list is selected.

To define a pre-selected option, add the **selected** attribute to the option:

```
<option value="fiat" selected>Fiat</option>
```

Visible Values:

Use the **size** attribute to specify the number of visible values:

```
<label for="cars">Choose a car:</label>
<select id="cars" name="cars" size="3">
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat">Fiat</option>
  <option value="audi">Audi</option>
</select>
```

Allow Multiple Selections:

Use the **multiple** attribute to allow the user to select more than one value:

```
<label for="cars">Choose a car:</label>
<select id="cars" name="cars" size="4" multiple>
  <option value="volvo">Volvo</option>
  <option value="saab">Saab</option>
  <option value="fiat">Fiat</option>
  <option value="audi">Audi</option>
</select>
```

The <textarea> Element

The <textarea> element defines a multi-line input field (a text area):

```
<textarea name="message" rows="10" cols="30">
The cat was playing in the garden.
</textarea>
```

The **rows** attribute specifies the visible number of lines in a text area.

The **cols** attribute specifies the visible width of a text area.

This is how the HTML code above will be displayed in a browser:

You can also define the size of the text area by using CSS:

```
<textarea name="message" style="width:200px; height:600px;">
The cat was playing in the garden.
</textarea>
```

The <button> Element

The <button> element defines a clickable button:

```
<button type="button" onclick="alert('Hello World!')">Click Me!</button>
```

The <fieldset> and <legend> Elements

The <fieldset> element is used to group related data in a form.

The <legend> element defines a caption for the <fieldset> element.

```
<form action="/action_page.php">
  <fieldset>
    <legend>Personalia:</legend>
    <label for="fname">First name:</label><br>
    <input type="text" id="fname" name="fname" value="John"><br>
    <label for="lname">Last name:</label><br>
    <input type="text" id="lname" name="lname" value="Doe"><br><br>
    <input type="submit" value="Submit">
  </fieldset>
</form>
```

The <datalist> Element

The <datalist> element specifies a list of pre-defined options for an <input> element.

Users will see a drop-down list of the pre-defined options as they input data.

The list attribute of the <input> element, must refer to the id attribute of the <datalist> element.

```
<form action="/action_page.php">
  <input list="browsers">
  <datalist id="browsers">
    <option value="Edge">
    <option value="Firefox">
    <option value="Chrome">
    <option value="Opera">
    <option value="Safari">
  </datalist>
</form>
```

Perform a calculation and show the result in an <output> element:

```
<form action="/action_page.php"
  oninput="x.value=parseInt(a.value)+parseInt(b.value)">
  0
  <input type="range" id="a" name="a" value="50">
  100 +
  <input type="number" id="b" name="b" value="50">
  =
  <output name="x" for="a b"></output>
  <br><br>
  <input type="submit">
</form>
```

HTML Input Types

Here are the different input types you can use in HTML:

- <input type="button">
- <input type="checkbox">
- <input type="color">
- <input type="date">

- `<input type="datetime-local">`
- `<input type="email">`
- `<input type="file">`
- `<input type="hidden">`
- `<input type="image">`
- `<input type="month">`
- `<input type="number">`
- `<input type="password">`
- `<input type="radio">`
- `<input type="range">`
- `<input type="reset">`
- `<input type="search">`
- `<input type="submit">`
- `<input type="tel">`
- `<input type="text">`
- `<input type="time">`
- `<input type="url">`
- `<input type="week">`

Input Type Text

`<input type="text">` defines a **single-line text input field**

```
<form>
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname">
</form>
```

Input Type Password

`<input type="password">` defines a **password field**:

```
<form>
<label for="username">Username:</label><br>
<input type="text" id="username" name="username"><br>
<label for="pwd">Password:</label><br>
<input type="password" id="pwd" name="pwd">
</form>
```

Input Type Submit

`<input type="submit">` defines a button for **submitting** form data to a **form-handler**.

The form-handler is typically a server page with a script for processing input data.

The form-handler is specified in the form's **action** attribute:

```
<form action="/action_page.php">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
</form>
```

If you omit the submit button's value attribute, the button will get a default text:

```
<form action="/action_page.php">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
```

```
<input type="submit">
</form>
```

Input Type Reset

`<input type="reset">` defines a **reset button** that will reset all form values to their default values:

```
<form action="/action_page.php">
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" value="John"><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname" value="Doe"><br><br>
<input type="submit" value="Submit">
<input type="reset" value="Reset">
</form>
```

Input Type Radio

`<input type="radio">` defines a **radio button**.

Radio buttons let a user select ONLY ONE of a limited number of choices:

`<p>Choose your favorite Web language:</p>`

```
<form>
<input type="radio" id="html" name="fav_language" value="HTML">
<label for="html">HTML</label><br>
<input type="radio" id="css" name="fav_language" value="CSS">
<label for="css">CSS</label><br>
<input type="radio" id="javascript" name="fav_language" value="JavaScript">
<label for="javascript">JavaScript</label>
</form>
```

Input Type Checkbox

`<input type="checkbox">` defines a **checkbox**.

Checkboxes let a user select ZERO or MORE options of a limited number of choices.

```
<form>
<input type="checkbox" id="vehicle1" name="vehicle1" value="Bike">
<label for="vehicle1"> I have a bike</label><br>
<input type="checkbox" id="vehicle2" name="vehicle2" value="Car">
<label for="vehicle2"> I have a car</label><br>
<input type="checkbox" id="vehicle3" name="vehicle3" value="Boat">
<label for="vehicle3"> I have a boat</label>
</form>
```

Input Type Button

`<input type="button">` defines a **button**:

```
<input type="button" onclick="alert('Hello World!')" value="Click Me!">
```

Input Type Color

The `<input type="color">` is used for input fields that should contain a color.

Depending on browser support, a color picker can show up in the input field.

```
<form>
<label for="favcolor">Select your favorite color:</label>
<input type="color" id="favcolor" name="favcolor">
```

```
</form>
```

Input Type Date

The `<input type="date">` is used for input fields that should contain a date. Depending on browser support, a date picker can show up in the input field.

```
<form>
<label for="birthday">Birthday:</label>
<input type="date" id="birthday" name="birthday">
</form>
```

You can also use the `min` and `max` attributes to add restrictions to dates:

```
<form>
<label for="datemax">Enter a date before 1980-01-01:</label>
<input type="date" id="datemax" name="datemax" max="1979-12-31"><br><br>
<label for="datemin">Enter a date after 2000-01-01:</label>
<input type="date" id="datemin" name="datemin" min="2000-01-02">
</form>
```

Input Type Datetime-local

The `<input type="datetime-local">` specifies a date and time input field, with no time zone. Depending on browser support, a date picker can show up in the input field.

```
<form>
<label for="birthdaytime">Birthday (date and time):</label>
<input type="datetime-local" id="birthdaytime" name="birthdaytime">
</form>
```

Input Type Email

The `<input type="email">` is used for input fields that should contain an e-mail address. Depending on browser support, the e-mail address can be automatically validated when submitted. Some smartphones recognize the email type, and add ".com" to the keyboard to match email input.

```
<form>
<label for="email">Enter your email:</label>
<input type="email" id="email" name="email">
</form>
```

Input Type Image

The `<input type="image">` defines an image as a submit button. The path to the image is specified in the `src` attribute.

```
<form>
<input type="image" src="img_submit.gif" alt="Submit" width="48" height="48">
</form>
```

Input Type File

The `<input type="file">` defines a file-select field and a "Browse" button for file uploads.

```
<form>
<label for="myfile">Select a file:</label>
<input type="file" id="myfile" name="myfile">
</form>
```

Input Type Hidden

The `<input type="hidden">` defines a hidden input field (not visible to a user).

A hidden field lets web developers include data that cannot be seen or modified by users when a form is submitted.

A hidden field often stores what database record that needs to be updated when the form is submitted.

Note: While the value is not displayed to the user in the page's content, it is visible (and can be edited) using any browser's developer tools or "View Source" functionality. Do not use hidden inputs as a form of security!

```
<form>
  <label for="fname">First name:</label>
  <input type="text" id="fname" name="fname"><br><br>
  <input type="hidden" id="custId" name="custId" value="3487">
  <input type="submit" value="Submit">
</form>
```

Input Type Month

The `<input type="month">` allows the user to select a month and year.

Depending on browser support, a date picker can show up in the input field.

```
<form>
  <label for="bdaymonth">Birthday (month and year):</label>
  <input type="month" id="bdaymonth" name="bdaymonth">
</form>
```

Input Type Number

The `<input type="number">` defines a **numeric** input field.

You can also set restrictions on what numbers are accepted.

The following example displays a numeric input field, where you can enter a value from 1 to 5:

```
<form>
  <label for="quantity">Quantity (between 1 and 5):</label>
  <input type="number" id="quantity" name="quantity" min="1" max="5">
</form>
```

Input Restrictions

Here is a list of some common input restrictions:

Attribute	Description
checked	Specifies that an input field should be pre-selected when the page loads (for type="checkbox" or type="radio")
disabled	Specifies that an input field should be disabled
max	Specifies the maximum value for an input field
maxlength	Specifies the maximum number of character for an input field
min	Specifies the minimum value for an input field
pattern	Specifies a regular expression to check the input value against
readonly	Specifies that an input field is read only (cannot be changed)
required	Specifies that an input field is required (must be filled out)
size	Specifies the width (in characters) of an input field
step	Specifies the legal number intervals for an input field
value	Specifies the default value for an input field

The following example displays a numeric input field, where you can enter a value from 0 to 100, in steps of 10. The default value is 30:

```
<form>
<label for="quantity">Quantity:</label>
<input type="number" id="quantity" name="quantity" min="0" max="100" step="10" value="30">
</form>
```

Input Type Range

The `<input type="range">` defines a control for entering a number whose exact value is not important (like a slider control). Default range is 0 to 100. However, you can set restrictions on what numbers are accepted with the `min`, `max`, and `step` attributes:

```
<form>
<label for="vol">Volume (between 0 and 50):</label>
<input type="range" id="vol" name="vol" min="0" max="50">
</form>
```

Input Type Search

The `<input type="search">` is used for search fields (a search field behaves like a regular text field).

```
<form>
<label for="gsearch">Search Google:</label>
<input type="search" id="gsearch" name="gsearch">
</form>
```

Input Type Tel

The `<input type="tel">` is used for input fields that should contain a telephone number.

```
<form>
<label for="phone">Enter your phone number:</label>
<input type="tel" id="phone" name="phone" pattern="[0-9]{3}-[0-9]{2}-[0-9]{3}">
</form>
```

Input Type Time

The `<input type="time">` allows the user to select a time (no time zone). Depending on browser support, a time picker can show up in the input field.

```
<form>
<label for="appt">Select a time:</label>
<input type="time" id="appt" name="appt">
</form>
```

Input Type Url

The `<input type="url">` is used for input fields that should contain a URL address. Depending on browser support, the url field can be automatically validated when submitted. Some smartphones recognize the url type, and adds ".com" to the keyboard to match url input.

```
<form>
<label for="homepage">Add your homepage:</label>
<input type="url" id="homepage" name="homepage">
</form>
```

Input Type Week

The `<input type="week">` allows the user to select a week and year.

Depending on browser support, a date picker can show up in the input field.

```
<form>
  <label for="week">Select a week:</label>
  <input type="week" id="week" name="week">
</form>
```

HTML Input Type Attribute

Tag	Description
<code><input type=""></code>	Specifies the input type to display

HTML Input Attributes

This chapter describes the different attributes for the HTML `<input>` element.

The value Attribute

The input **value** attribute specifies an initial value for an input field:

Input fields with initial (default) values:

```
<form>
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John"><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe">
</form>
```

The readonly Attribute

The input **readonly** attribute specifies that an input field is read-only.

A read-only input field cannot be modified (however, a user can tab to it, highlight it, and copy the text from it).

The value of a read-only input field will be sent when submitting the form!

```
<form>
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John" readonly><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe">
</form>
```

The disabled Attribute

The input **disabled** attribute specifies that an input field should be disabled.

A disabled input field is unusable and un-clickable.

The value of a disabled input field will not be sent when submitting the form!

```
<form>
  <label for="fname">First name:</label><br>
  <input type="text" id="fname" name="fname" value="John" disabled><br>
  <label for="lname">Last name:</label><br>
  <input type="text" id="lname" name="lname" value="Doe">
</form>
```

The size Attribute

The input **size** attribute specifies the visible width, in characters, of an input field.

The default value for **size** is 20.

Note: The **size** attribute works with the following input types: text, search, tel, url, email, and password.

```
<form>
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" size="50"><br>
<label for="pin">PIN:</label><br>
<input type="text" id="pin" name="pin" size="4">
</form>
```

The maxlength Attribute

The input **maxlength** attribute specifies the maximum number of characters allowed in an input field.

Note: When a **maxlength** is set, the input field will not accept more than the specified number of characters. However, this attribute does not provide any feedback. So, if you want to alert the user, you must write JavaScript code.

```
<form>
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" size="50"><br>
<label for="pin">PIN:</label><br>
<input type="text" id="pin" name="pin" maxlength="4" size="4">
</form>
```

The min and max Attributes

The input **min** and **max** attributes specify the minimum and maximum values for an input field.

The **min** and **max** attributes work with the following input types: number, range, date, datetime-local, month, time and week.

Tip: Use the max and min attributes together to create a range of legal values.

Set a max date, a min date, and a range of legal values:

```
<form>
<label for="datemax">Enter a date before 1980-01-01:</label>
<input type="date" id="datemax" name="datemax" max="1979-12-31"><br><br>

<label for="datemin">Enter a date after 2000-01-01:</label>
<input type="date" id="datemin" name="datemin" min="2000-01-02"><br><br>

<label for="quantity">Quantity (between 1 and 5):</label>
<input type="number" id="quantity" name="quantity" min="1" max="5">
</form>
```

The multiple Attribute

The input **multiple** attribute specifies that the user is allowed to enter more than one value in an input field.

The **multiple** attribute works with the following input types: email, and file.

```
<form>
<label for="files">Select files:</label>
<input type="file" id="files" name="files" multiple>
</form>
```

The pattern Attribute

The input **pattern** attribute specifies a regular expression that the input field's value is checked against, when the form is submitted.

The **pattern** attribute works with the following input types: text, date, search, url, tel, email, and password.

Tip: Use the global [title](#) attribute to describe the pattern to help the user.

Tip: Learn more about [regular expressions](#) in our JavaScript tutorial.

```
<form>
<label for="country_code">Country code:</label>
<input type="text" id="country_code" name="country_code"
  pattern="[A-Za-z]{3}" title="Three letter country code">
</form>
```

The placeholder Attribute

The input **placeholder** attribute specifies a short hint that describes the expected value of an input field (a sample value or a short description of the expected format).

The short hint is displayed in the input field before the user enters a value.

The **placeholder** attribute works with the following input types: text, search, url, number, tel, email, and password.

```
<form>
<label for="phone">Enter a phone number:</label>
<input type="tel" id="phone" name="phone"
  placeholder="123-45-678"
  pattern="[0-9]{3}-[0-9]{2}-[0-9]{3}">
</form>
```

The required Attribute

The input **required** attribute specifies that an input field must be filled out before submitting the form.

The **required** attribute works with the following input types: text, search, url, tel, email, password, date pickers, number, checkbox, radio, and file.

```
<form>
<label for="username">Username:</label>
<input type="text" id="username" name="username" required>
</form>
```

The step Attribute

The input **step** attribute specifies the legal number intervals for an input field.

Example: if step="3", legal numbers could be -3, 0, 3, 6, etc.

Tip: This attribute can be used together with the max and min attributes to create a range of legal values.

The **step** attribute works with the following input types: number, range, date, datetime-local, month, time and week.

```
<form>
<label for="points">Points:</label>
<input type="number" id="points" name="points" step="3">
</form>
```

The autofocus Attribute

The input **autofocus** attribute specifies that an input field should automatically get focus when the page loads.

```
<form>
<label for="fname">First name:</label><br>
<input type="text" id="fname" name="fname" autofocus><br>
<label for="lname">Last name:</label><br>
<input type="text" id="lname" name="lname">
</form>
```

The height and width Attributes

The input **height** and **width** attributes specify the height and width of an `<input type="image">` element.

```
<form>
<label for="fname">First name:</label>
<input type="text" id="fname" name="fname"><br><br>
<label for="lname">Last name:</label>
<input type="text" id="lname" name="lname"><br><br>
<input type="image" src="img_submit.gif" alt="Submit" width="48" height="48">
</form>
```

The list Attribute

The input **list** attribute refers to a `<datalist>` element that contains pre-defined options for an `<input>` element.

```
<form>
<input list="browsers">
<datalist id="browsers">
  <option value="Edge">
  <option value="Firefox">
  <option value="Chrome">
  <option value="Opera">
  <option value="Safari">
</datalist>
</form>
```

The autocomplete Attribute

The input **autocomplete** attribute specifies whether a form or an input field should have autocomplete on or off.

Autocomplete allows the browser to predict the value. When a user starts to type in a field, the browser should display options to fill in the field, based on earlier typed values.

The **autocomplete** attribute works with `<form>` and the following `<input>` types: text, search, url, tel, email, password, datepickers, range, and color.

```
<form action="/action_page.php" autocomplete="on">
<label for="fname">First name:</label>
<input type="text" id="fname" name="fname"><br><br>
<label for="lname">Last name:</label>
<input type="text" id="lname" name="lname"><br><br>
<label for="email">Email:</label>
<input type="email" id="email" name="email" autocomplete="off"><br><br>
<input type="submit" value="Submit">
</form>
```

HTML Form and Input Elements

Tag	Description
<form>	Defines an HTML form for user input
<input>	Defines an input control

Learn CSS

CSS Introduction

CSS is the language we use to style an HTML document.

CSS describes how HTML elements should be displayed.

What is CSS?

CSS is the language we use to style a Web page.

- CSS stands for Cascading Style Sheets
- CSS describes how HTML elements are to be displayed on screen, paper, or in other media
- CSS saves a lot of work. It can control the layout of multiple web pages all at once
- External stylesheets are stored in CSS files

Why Use CSS?

CSS is used to define styles for your web pages, including the design, layout and variations in display for different devices and screen sizes.

CSS Example

```
body {  
  background-color: lightblue;  
}  
  
h1 {  
  color: white;  
  text-align: center;  
}  
  
p {  
  font-family: verdana;  
  font-size: 20px;  
}
```

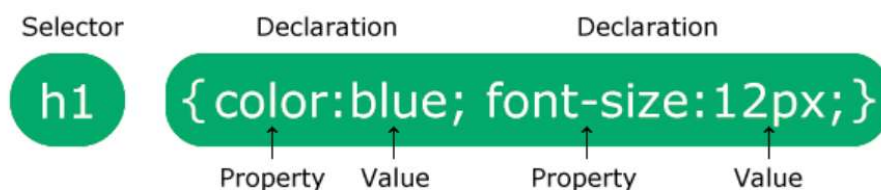
CSS Saves a Lot of Work!

The CSS definitions are normally saved in an external .css file.

With an external stylesheet file, you can change the look of an entire website by changing just one file!

CSS Syntax

A CSS rule consists of a selector and a declaration block:



The selector points to the HTML element you want to style.

The declaration block contains one or more declarations separated by semicolons.

Each declaration includes a CSS property name and a value, separated by a colon.

Multiple CSS declarations are separated with semicolons, and declaration blocks are surrounded by curly braces.

Example:

```
p {  
  color: red;  
  text-align: center;  
}
```

Example Explained

- `p` is a selector in CSS (it points to the HTML element you want to style: `<p>`).
- `color` is a property, and `red` is the property value
- `text-align` is a property, and `center` is the property value

CSS Selectors

CSS selectors are used to "find" (or select) the HTML elements you want to style.

We can divide CSS selectors into five categories:

- Simple selectors (select elements based on name, id, class)
- [Combinator selectors](#) (select elements based on a specific relationship between them)
- [Pseudo-class selectors](#) (select elements based on a certain state)
- [Pseudo-elements selectors](#) (select and style a part of an element)
- [Attribute selectors](#) (select elements based on an attribute or attribute value)

The CSS element Selector

The [element](#) selector selects HTML elements based on the element name.

Example

Here, all `<p>` elements on the page will be center-aligned, with a red text color:

```
p {  
  text-align: center;  
  color: red;  
}
```

The CSS id Selector

The [id](#) selector uses the id attribute of an HTML element to select a specific element.

The id of an element is unique within a page, so the id selector is used to select one unique element!

To select an element with a specific id, write a hash (`#`) character, followed by the id of the element.

Example

The CSS rule below will be applied to the HTML element with `id="para1"`:

```
#para1 {  
  text-align: center;  
  color: red;  
}
```

The CSS class Selector

The [class](#) selector selects HTML elements with a specific class attribute.

To select elements with a specific class, write a period (`.`) character, followed by the class name.

Example

In this example all HTML elements with `class="center"` will be red and center-aligned:

```
.center {
  text-align: center;
  color: red;
}
```

You can also specify that only specific HTML elements should be affected by a class.

Example

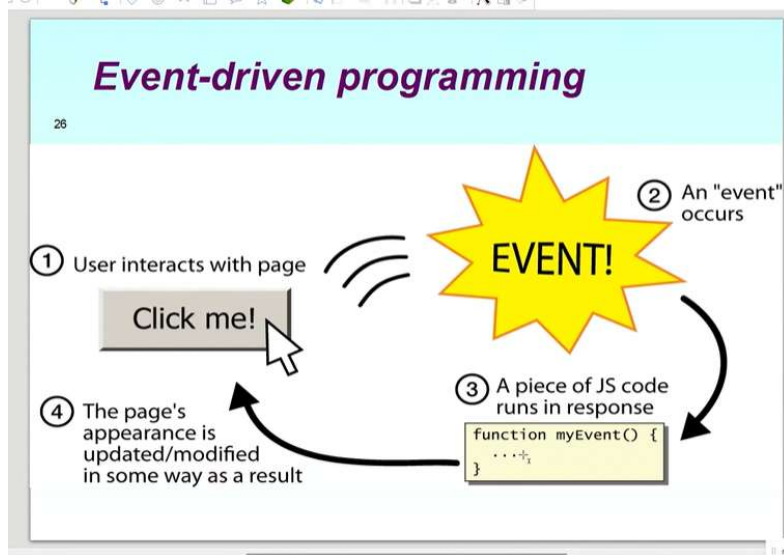
In this example only <p> elements with class="center" will be red and center-aligned:

```
p.center {
  text-align: center;
  color: red;
}
```

From <https://www.w3schools.com/css/css_selectors.asp>

=====

Event-Driven programming is important for Exam point of view



Screen clipping taken: 22-08-2026 18:58

Events

- onsubmit - call when submit button is clicked
- onclick - call when this button is clicked
- onreset - call when the reset button is clicked
- onload - call after page loads
- onmouseover - call when mouse pointer enters image area
- onmouseout - call when mouse pointer leaves image area
- onfocus - call when control receives focus
- onblur - call when a control loses focus
- onchange - call when a control loses focus and the value of its contents has changed
- many more

Screen clipping taken: 22-08-2026 19:00

Keyboard, load event,

Object	Event Handlers			
window	onload	onunload	onblur	onfocus
link	onclick	onmouseout	onmouseover	
image	onabort	onerror	onload	
form	onreset	onsubmit		
text	password	onblur	onchange	onfocus
textarea	onblur	onchange	onfocus	
button	onclick			
reset	onclick			
submit	onclick			
radio	onclick			
checkbox	onclick			
select	onblur	onchange	onfocus	
fileupload	onblur	onchange	onfocus	

Screen clipping taken: 22-08-2026 19:23

JavaScript functions

30

```
function name() {  
  statement ;  
  statement ;  
  ...  
  statement ;  
}
```

```
function myFunction() {  
  alert("Hello!");  
  alert("How are you?");  
}
```

- the above could be the contents of `example.js` linked to our HTML page
- statements placed into functions can be evaluated in response to user events

Screen clipping taken: 22-08-2026 19:27

JavaScript is not Java

- By now you should have realized that you *already know* a great deal of JavaScript
 - So far we have talked about things that are *the same* as in Java
- JavaScript has some features that *resemble* features in Java:
 - JavaScript has Objects and primitive data types
 - JavaScript has qualified names; for example, `document.write("Hello World");`
 - JavaScript has Events and event handlers
 - Exception handling in JavaScript is *almost* the same as in Java
- JavaScript has some features *unlike* in Java:
 - *Variable names* are untyped: the type of a variable depends on the value it is currently holding
 - Objects and arrays are defined in quite a different way

31

Screen clipping taken: 22-08-2026 19:27

JavaScript Statements

```
<html>
<head><title>My Page</title></head>
<body>
<script language="JavaScript">

document.write('<h1>This is my first →
JavaScript Page</h1>');

</script>
</body>
</html>
```

HTML written inside JavaScript

Screen clipping taken: 22-08-2026 19:29

JavaScript Statements

```
<html>
<head><title>My Page</title></head>
<body>
<p>
<a href="myfile.html">My Page</a>
<br />
<a href="myfile.html"
onmouseover="window.alert('Hello');">
My Page</A>
</p>
</body>
</html>
```

An Event

JavaScript written inside HTML



Screen clipping taken: 22-08-2026 19:29

Example Statements

```
<script language="JavaScript">  
window.prompt('Enter your name:', '');  
</script>
```

```
<form>  
<input type="button" Value="Press"  
  onClick="window.alert('Hello');">  
</form>
```

Another event

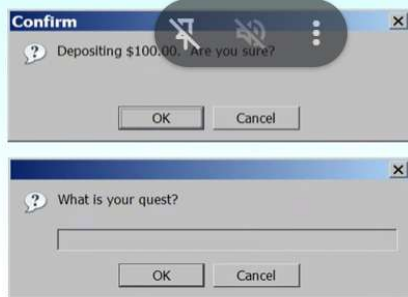
Note quotes: " and '

Screen clipping taken: 22-08-2026 19:31

Popup boxes

37

```
alert("message"); // message  
confirm("message"); // returns true or false  
prompt("message"); // returns user input string  
JS
```



Screen clipping taken: 22-08-2026 19:32

Forms and JavaScript

document.formname.elementname.value
Thus:

```
document.addressform.yourname.value  
document.addressform.phone.value  
document.addressform.email.value
```

The image shows a form with three input fields. The first field is labeled 'Name:', the second 'Phone:', and the third 'Email:'. Each field has a small toolbar with a close button (X) and a help button (?) in the top right corner. The 'Phone' field has a small '+' icon in the center.

Screen clipping taken: 22-08-2026 19:35

Using Form Data

Personalising an alert box

Enter your name:



```
<form name="alertform">
Enter your name:
<input type="text" name="yourname">
<input type="button" value="Go"
  onClick="window.alert('Hello ' + 
    document.alertform.yourname.value); ">
</form>
```

Screen clipping taken: 22-08-2026 19:39

```
<html>
<script language="JavaScript">
var ans = 0;
var firstnum = 0;
var secondnum = 0;
firstnum = prompt("Enter the first number",0);
secondnum = prompt("Enter the second number",0);
ans = firstnum * secondnum;
document.write(ans);
</script>
</html>
```

The JavaScript var (for variable) can be used to define a field (note that you can eliminate the command var and it will still work). In this example, I have defined three variables and given them all an initial value of 0.

The prompt can be used to take in data. The message in quotes is used to ask the user for input and the 0 indicates it is numeric input. Note that the data that was keyed in with the first prompt was assigned to firstnum and the data that was keyed in with the second prompt was assigned to secondnum.

ans = firstnum * secondnum;
This takes the two numbers stored in firstnum and secondnum and multiplies them together. The results are stored in ans.

document.write(ans);
I am now using the write method to put ans to the screen. Note that ans is a variable, not a literal so it is not enclosed in quotes.

Screen clipping taken: 22-08-2026 19:41

```
<html>
<script language="JavaScript">
var data_input = 0;
data_input=prompt("How many times should I write the line?",0);
var ct=1;
while (ct <= data_input)
{
  document.write("This is number " + ct + "<br>");
  ct = ct + 1;
}
document.write("<br>" + "This is the end!!!")
</script>
</html>
```

The while says to do this loop while the contents of the ct variable is less then or equal to the contents of the data_input variable. Note that the data_input variable contains user input. The ct variable is controlled by the program.

The + means concatenation so I am putting the three things together in the document.write

Inside the loop, I increment ct by 1 using the structure ct = ct + 1. This means that eventually ct will no longer be less than or equal to the number that the user inputted so the loop will end.

When the loop ends the message, This is the end is written. Note that the
 means that it will be written after moving to a new line.

Screen clipping taken: 22-08-2026 19:45

```

<html>
<script language="JavaScript">
monthArray = new Array(12);
monthArray[1]="January";
monthArray[2]="February";
monthArray[3]="March";
monthArray[4]="April";
monthArray[5]="May";
monthArray[6]="June";
monthArray[7]="July";
monthArray[8]="August";
monthArray[9]="September";
monthArray[10]="October";
monthArray[11]="November";
monthArray[12]="December";
var user_month = 0;
var user_day = 0;
var user_year = 0;
user_month = prompt("Enter the month as a number",0);
user_day = prompt("Enter the day",0);
user_year = prompt("Enter the year",0);
document.write("The date is " + monthArray[user_month] + " " + user_day + ", " + user_year);
</script>
</html>

```

Here I have defined an array and filled the array - I decided not to use the 0th element even though it would be available with this definition.

Now I am taking in user input for the month, day and year.

Now I am concatenating the literals, spaces and parts of the date together to produce the output. The [] holds the index/subscript/pointer which selects which element of the array I want. The elements are named monthArray.

Screen clipping taken: 22-08-2026 19:46

User input/output

57

```

<SCRIPT TYPE="text/javascript">
var firstNumber, // first string entered by user
secondNumber, // second string entered by user
number1, // first number to add
number2, // second number to add
sum; // sum of number1 and number2
// read in first number from user as a string
firstNumber = window.prompt("Enter first integer", "0");
// read in second number from user as a string
secondNumber = window.prompt("Enter second integer", "0");
// convert numbers from strings to integers
firstNumber = parseInt(firstNumber);
number2 = parseInt(secondNumber);
// add the numbers
sum = firstNumber + number2;
// display the results
document.writeln("<H1>The sum is " + sum + "</H1>");
</SCRIPT>

```

Screen clipping taken: 22-08-2026 19:47

Functions

58

```

<SCRIPT TYPE="text/javascript">
var input1 = window.prompt("Enter first number", "0");
var input2 = window.prompt("Enter second number", "0");
var input3 = window.prompt("Enter third number", "0");
var value1 = parseFloat(input1);
var value2 = parseFloat(input2);
var value3 = parseFloat(input3);
var maxValue = maximum(value1, value2, value3);
document.writeln("First number: " + value1 + "<br>");
document.writeln("Second number: " + value2 + "<br>");
document.writeln("Third number: " + value3 + "<br>");
document.writeln("Maximum is: " + maxValue);

// maximum method definition (called from above)
function maximum(x, y, z) {
return Math.max(x, Math.max(y, z));
}
</SCRIPT>

```